

Interview Task 006: Design System Components - Candidate Brief

Role: Junior Developer (React)

Company: Intrahealth (a subsidiary of HEALWELL AI)

Format: Take-home exercise + pre-recorded walkthrough + technical interview

Time budget: ~4 hours (take-home) + 15–20 min recording

The Challenge

You are joining a healthcare software company that builds an Electronic Health Record (EHR) system. The frontend team is building a React design system that several product teams consume, and moving designs from a prototyping tool into that system.

Your task has four parts:

1. **Build a component library** of five components from the design spec below.
2. **Document it** so that another developer could use it without asking you anything.
3. **Build two pages of a website** in React using only that library: a patient list and a patient detail view.
4. **Stand up a C# mock API** that the website reads from.

The library, the website, and the API are separate packages. Whether they live in one repo or three is up to you.

To be clear: we are not asking for a large component set. Five components built carefully, documented well, proven by two pages that use them, and backed by an API that returns fake data is the whole task.

What Is a Design System Component?

A design system component is a reusable building block that other developers assemble into screens. Three things make it one:

- **A small, well-defined set of props.** The consumer sets a variant or a label; they never pass in a colour or a pixel value.
- **It handles its own states.** Hover, focus, error, loading, and disabled are the component's job, not the page's.

- **It is accessible by default.** Labels are attached to inputs, buttons work from the keyboard, and disabled things are actually disabled.

The person consuming it should not need to read its source to use it correctly.

Think of it as the difference between "I styled a button on this page" and "I built a Button that forty pages can use without any of them needing to know how it works inside." The documentation is what makes the second one possible.

What We Provide

- The design spec below: values, component props, page contents, and the API contract. It is complete and is the only design input you get. There are no screenshots. If something is not specified, it is not required, and any reasonable choice is fine.

The library and the website must be React and TypeScript. The API must be C# on ASP.NET Core. Everything else (styling approach, test runner, build tooling, routing, .NET version, minimal API or controllers) is yours.

Token Sheet

Name	Value
color.primary	#1F6FEB
color.primary.hover	#185CC4
color.danger	#C93C37
color.text	#1A1A1A
color.text.muted	#6B7280
color.border	#D1D5DB
color.focus	#93C5FD
color.surface	FFFFFF
color.surface.subtle	#F3F4F6
color.disabled.bg	#E5E7EB
color.disabled.text	#9CA3AF
space.1	4px

space.2	8px
space.3	12px
space.4	16px
space.6	24px
space.8	32px
radius.sm	4px
radius.md	8px
font.body	14px / 20px
font.label	12px / 16px
font.button	14px / 20px, weight 600
font.heading	20px / 28px, weight 600
font.title	24px / 32px, weight 600

The component and page sections below refer to these names.

Part 1: The Component Library (Your Work Order)

Package name: `ui`. React and TypeScript. Five components, each exported from the package entry point.

Button

A clickable button that triggers an action. Comes in two visual styles (primary for the main action on a screen, secondary for everything else), two sizes, and can show that it is busy or unavailable.

Prop	Type	Required	Default
variant	<code>"primary"</code> <code>"secondary"</code>	No	<code>"primary"</code>
size	<code>"sm"</code> <code>"md"</code>	No	<code>"md"</code>
loading	<code>boolean</code>	No	<code>false</code>

disabled	<code>boolean</code>	No	<code>false</code>
onClick	<code>() => void</code>	No	
children	<code>ReactNode</code>	Yes	

- Primary: color.primary background, white text, color.primary.hover on hover.
- Secondary: color.surface background, color.border border, color.text text, color.surface.subtle on hover.
- sm: space.1 vertical padding, space.3 horizontal. md: space.2 vertical, space.4 horizontal. Both radius.sm, font.button.
- Loading: spinner replaces the label, button is non-interactive, width does not change.
- Disabled: color.disabled.bg background, color.disabled.text text, non-interactive.
- Focus: 2px outline in color.focus, 2px offset.

TextField

A single-line text input with a label above it. It can show a short hint below the input, or an error message when the value is invalid.

Prop	Type	Required	Default
label	<code>string</code>	Yes	
value	<code>string</code>	Yes	
onChange	<code>(value: string) => void</code>	Yes	
placeholder	<code>string</code>	No	
helperText	<code>string</code>	No	
errorMessage	<code>string</code>	No	
disabled	<code>boolean</code>	No	<code>false</code>

- Label above the input in font.label, color.text.muted, space.1 gap.
- Input: space.2 vertical padding, space.3 horizontal, color.border border, radius.sm, font.body.
- Focus: border becomes color.primary, 2px outline in color.focus.
- Helper text below the input in font.label, color.text.muted, space.1 gap.

- Error state: the field is in error state when `errorMessage` is provided and not empty. The border becomes `color.danger` and the message is shown below the input in `color.danger`, replacing the helper text. Otherwise the field is not in error state and the helper text, if provided, shows.
- Disabled: `color.disabled.bg` background, `color.disabled.text` text.
- Label must be associated with the input.

Card

A bordered container that groups related content on a page. It has an optional title at the top and an optional slot on the same row for buttons; whatever you place inside it becomes the body.

Prop	Type	Required	Default
title	<code>string</code>	No	
actions	<code>ReactNode</code>	No	
children	<code>ReactNode</code>	Yes	

- `color.surface` background, `color.border` border, `radius.md`, `space.6` padding.
- Title in `font.heading` at the top, actions right-aligned on the same row, `space.4` gap below the row.

Table

A data table: a header row of column names and one body row per record. Rows can optionally be clickable, and it shows a message when there are no rows.

Prop	Type	Required	Default
columns	<code>{ key: string; header: string } []</code>	Yes	
rows	<code>Record<string, ReactNode> []</code>	Yes	
onRowClick	<code>(row) => void</code>	No	
emptyMessage	<code>string</code>	No	<code>"No results"</code>

- Header row: font.label, color.text.muted, color.surface.subtle background.
- Body rows: font.body, space.3 vertical padding, space.4 horizontal, color.border bottom border.
- Row hover when onRowClick is set: color.surface.subtle background, pointer cursor.
- Empty state: emptyMessage centred in color.text.muted with space.8 vertical padding.

DescriptionList

A read-only list of label and value pairs, one per row, for showing the details of a single record (for example a patient's name, date of birth, and phone). Think of the rows on an ID card. It is the read-only counterpart to a form.

Prop	Type	Required	Default
items	<pre>{ label: string; value: ReactNode }[]</pre>	Yes	

- One row per item: label in font.label, color.text.muted, fixed width 160px; value in font.body, color.text, to its right.
- space.3 vertical gap between rows, color.border bottom border on each row except the last.
- An item whose value is empty, null, or undefined renders the value as "—" in color.text.muted.

Part 2: The Documentation

The documentation lives in the `ui` package and is graded as carefully as the code. It must let a developer who has never seen the library build a new page without reading the source or asking you anything. Required sections:

1. **Getting started.** How to install and consume `ui` from another package or repo.
2. **Styling.** How the values from the Token Sheet are defined and used, so a new component can match the existing ones.
3. **Component reference.** For each component: a props table (name, type, default, description), one usage example, and a short note on when to use it and when not to.
4. **Contributing.** How to add a sixth component so that it fits with the existing five (file layout, export, styling, tests).

Storybook or similar stories are optional. They count as usage examples, not as documentation on their own.

Part 3: The Mock API

A C# ASP.NET Core project that serves the patient data the website needs. It is a mock: in-memory data you invent, no database, no authentication. Design the endpoints as you see fit for the website described in Part 4. You can use FHIR.

Part 4: The Website

Package name: `app`. React and TypeScript. Two pages, built only from `ui` components and plain layout markup, reading from the API. `app` imports from the `ui` package entry point, does not override `ui` styles, and does not import from inside the `ui` package.

Page 1: Patient List

A clinic staff member opens the site and sees a list of patients.

- The page is titled "Patients". At the top is a Card with a single search field and a Search button. Below it is a Table of patients with columns Name, Gender, Birth date, Phone.
- When the page opens, the list loads from the API. While it loads, the user sees "Loading..." instead of the Table.
- The user types part of a name and clicks Search; the Table updates to show only matching patients. While the search runs, the Search button shows its loading state. Search runs on the button click; pressing Enter in the field is not required.
- If no patients match, the Table shows "No patients match your search".
- If the API cannot be reached, the user sees a Card titled "Something went wrong" instead of the Table.
- Clicking a patient row opens Page 2 for that patient.

Page 2: Patient Detail

The staff member has clicked a patient and sees that patient's demographics.

- A Back button returns to Page 1. The page is titled with the patient's full name.
- A Card titled "Demographics" shows a DescriptionList with Name, Gender, Birth date, Phone, Email, Address.
- While the patient loads, the user sees "Loading...".
- A missing value (for example a patient with no email) displays as "—", not blank and not "undefined".
- If the patient does not exist, the user sees a Card titled "Patient not found" and the Back button.

Layout, routing, and how the pages talk to the API are yours to decide.

What You Deliver

1. **The code.** Links to the GitHub repo or repos containing the library, the website, and the API. One repo with workspaces or three separate repos is your call; what matters is that `app` consumes `ui` as a package, not as a folder of source files. Each package must run with a documented command (for example `dotnet run` for the API and `npm install && npm run dev` for `app`).
2. **Your process.** However you worked, show it: the GitHub Issues or tickets you wrote for yourself, and your harness if you built one. Push up your `.claude` folder (or the equivalent for your tool), your skills, and any agent configuration. Write about it in the interview notes if you would like.
3. **Tests**, if you wrote any. Useful ones would cover Button variants and disabled state, TextField error state, Table empty state, DescriptionList rendering "—" for a missing value, and Page 1 rendering the Table from a mocked list response.
4. **The documentation.** As described in Part 2, inside the `ui` package.
5. **A project README.** A normal README for the project: what it is, how it is laid out, and how to run each package. Written as if the project were real, not as if it were an interview. It links to the interview notes below.
6. **Interview notes.** A separate markdown file (for example `INTERVIEW.md`) for everything specific to this exercise:
 - **Decisions.** Anything you chose that the spec left to you (one repo or several, styling approach, workspace tooling, routing, endpoint design, how `app` calls the API) and why.
 - **Process and AI usage.** Which tools you used, how you decomposed the work, what you delegated to the tools, and at least one thing you had to check or correct. Point to the tickets and harness from item 2.
 - **What I would do with two more hours.** What you would change, add, or fix.
7. **A pre-recorded walkthrough (15–20 min).** A video of you presenting the work: the running pages, the library, the documentation, and the decisions you made. Screen recording with voice is fine. We care about clarity of explanation. If we proceed, the follow-up technical interview opens with a short live demo from you and then moves to questions about your implementation and other technical questions.

The Four Areas

Your work should address these areas. Depth matters more than breadth.

1. Documentation and Communication

- Could a developer build a third page from your documentation alone?
- Does the documentation match the code exactly? Undocumented props and documented props that don't exist both count against you.
- Are the usage examples runnable as written?

2. Library Design

- Are props typed in a way that helps the consumer (union types for variant and size, not `string`)?
- Do the components use the Token Sheet values where the spec says?
- Are the label, focus, disabled, and loading behaviours actually implemented, not only styled?
- Does the package have a single clean entry point?

3. Consumption and Integration

- Does `app` use only the `ui` entry point, with no style overrides and no deep imports?
- Do the two pages reuse the components as specified, or introduce one-off markup that should have been a component?
- Is the API-to-display mapping in one place, and does it handle missing fields?
- Does search, navigation, loading, and error handling work as specified against the running API?
- Does the API serve what the website needs, and does the website behave as described in Part 4?

4. Process, Testing, and Repo Hygiene

- Does your process record show how you broke the work down and in what order, and does it match what the commit history shows?
- If you wrote tests, do they check behaviour rather than snapshot everything?
- Is the commit history readable?
- Does each package run with its documented command?

A Note on Tooling and AI Usage

We explicitly want to see how you use AI tools during this exercise, and we want you to tell us about it. Use Claude Code, Copilot, ChatGPT, Cursor, or whatever you normally work with.

We will ask you to explain code in your repo regardless of who wrote it. A candidate who can say "the tool generated this, I checked it by doing X, and I changed Y because Z" is doing exactly what we want.

What We're NOT Evaluating

- Visual polish beyond what the spec describes
 - Animation or transitions
 - Responsive or mobile layout
 - One repo or several, or which styling library, router, or workspace tool you chose
 - FHIR conformance or FHIR knowledge
 - Publishing the library to a registry
-

Getting Started

Some practical guidance for your 4 hours:

- **Decompose the work before you start.** Write the tickets you would want to pick up yourself: small, well-written, with a clear done state. A good breakdown makes the next four hours much easier and is part of what we assess.
- **Consider building a harness.** If you use AI tools, give them what they need to do good work: skills, the design spec, the documentation you have written so far, conventions for the repo. Time spent on this early tends to pay for itself.
- **Time-box anything that fights you.** Leave a note in the interview notes and move on.

Questions? If anything about the expectations is unclear, ask. We'd rather you spend time on the work than guessing what we want.

Good luck. We look forward to seeing your approach.